

V2X 스마트 지팡이
5.17 5 차 통합 미팅 수행계획서

하드웨어 마무리 / 팀원별 역할 1 부터 10 까지 / Jetson 배터리 연결 포함 / 현장 실행용 완전 상세본

항목	내용
회의 날짜	2026.05.17
예상 작업 시간	약 6 시간을 기본으로 하되, 하드웨어 마무리와 증거 확보가 끝날 때까지 확장 가능
핵심 목표	송신기 - 수신기 - Jetson - CSV 저장 - risk 출력 - 진동/음성 피드백까지 한 흐름으로 검증
이번 문서의 사용법	현장 체크리스트처럼 위에서 아래로 따라가며 실행한다. 모르는 부분은 “누가, 무엇을, 어떤 순서로, 어떤 증거를 남기는가”만 보고 움직이면 되도록 작성했다.
주의	이번 회차는 만능기판 납땜이나 완제품 매립이 아니라, 하드웨어 v0/v1 로 묶기 전 마지막 연결 검증이다.

문서 목차

- 0. 이번 수정본에서 바뀐 핵심
- 1. 5 차 미팅의 정확한 목표와 성공 기준
- 2. 현장 전체 운영 방식과 시간표
- 3. 팀원별 역할 총괄표 - 누가 무엇을 끝내야 하는가
- 4. 박중선 파트 - 하드웨어 재현, Jetson 배터리 연결, 출력부, 외관 정리
- 5. 강현준 파트 - 전체 연결성, Jetson 수신, CSV 저장, risk replay, GitHub
- 6. 최민서 파트 - SUMO/CSV 위험 시나리오, risk 계산, 시각화
- 7. End-to-End 통합 시연 절차
- 8. 코드 묶음
- 9. 문제 해결표
- 10. 최종 산출물 및 다음 회차 연결

0. 이번 수정본에서 바뀐 핵심

- 기존 문서에서 코드와 흐름은 어느 정도 잡혔지만, “누가 무엇을 어떻게 할지”가 부족했기 때문에 팀원별 작업을 현장 지시서 수준으로 다시 짰다.
- 박중선 파트에 Jetson 배터리 연결을 명확히 추가했다. 박중선은 출력부만 보는 것이 아니라, 하드웨어 전원 안정성 담당으로서 Jetson 전원까지 같이 확인한다.
- 각 작업마다 시작 조건, 실제 행동, 성공 기준, 사진/영상/로그 증거를 붙였다.
- 핀 연결은 “무조건 이 핀”이라고 외우기보다, 실제 보드 실크 인쇄와 코드에서 설정한 GPIO 가 일치하는지 확인하는 절차까지 포함했다.
- 5 차 당일 시간이 길어졌을 때 할 확장 작업과, 시간이 부족할 때 최소 성공 기준을 분리했다.

수정 요구	반영 내용
누가 무엇을 어떻게가 너무 적음	팀원별로 시작 전 준비, 실물 조작 순서, 완료 기준, 실패 시 전환 기준까지 추가
박중선 파트에 Jetson 배터리 연결 필요	박중선을 하드웨어 전원 안정성 담당으로 재정의하고, J48 점퍼, DC Barrel Jack, USB-PD 보조배터리, 금지 연결까지 절차화
PDF 만 보고 하드웨어 마무리 가능해야 함	회로, 포트, 코드, CSV, 촬영, GitHub, 산출물 백업까지 전부 실행 카드로 작성
길이 제한 없이 자세히	체크리스트, 역할표, 코드, 문제 해결표, 발표 멘트, 다음 회차 연결까지 확장

1. 5 차 미팅의 정확한 목표와 성공 기준

이번 5 차 미팅의 본질은 “새로운 기능을 많이 추가하는 날”이 아니라 “이미 따로 성공한 기능을 하나의 흐름으로 묶는 날”이다. 즉, 송신기와 수신기가 따로 되고, 진동/음성이 따로 되고, SUMO/CSV 가 따로 되는 상태에서 멈추면 발표용 시스템이 아니다. 5 차의 목표는 연결 검증이다.

구분	5 차에서 실제로 해야 하는 일	끝났다고 보는 기준
송신기	GPS/IMU 또는 테스트 패킷을 ESP32 에서 생성하고 ESP-NOW 로 보낸다.	Serial Monitor 에서 [SEND] seq 가 1 분 이상 증가한다.
수신기	ESP-NOW 패킷을 받고, 필요하면 Jetson 으로 JSON 또는 risk 명령을 전달한다.	[RX] seq 가 끊기지 않고 증가한다.

Jetson	수신 데이터를 읽고 CSV 로 저장하거나, 민서 CSV 를 replay 하여 risk 명령을 출력부로 보낸다.	터미널에 seq/risk 가 보이고 v2x_uart_log.csv 줄 수가 증가한다.
출력부	risk 0/1/2/3 에 따라 진동과 음성이 다르게 반응한다.	risk 단계별 영상이 남는다.
외관	디버깅 가능한 상태를 유지하면서도 선 빠짐을 줄이고 발표용으로 정리한다.	고정판/박스/라벨링 사진이 남는다.
증거	성공했다는 말이 아니라 영상, 사진, CSV, 코드 파일, README 로 남긴다.	1 분 이상 통합 영상과 최종 폴더가 존재한다.

1.1 최소 성공 / 중간 성공 / 대성공

등급	반드시 달성해야 하는 것	다음 회차로 넘겨도 되는 것
최소 성공	송신기 [SEND], 수신기 [RX], Jetson 또는 PC 에서 seq 로그, CSV 저장 중 하나 이상 확보	GPS fix, 외관 박스, Transformer 학습
중간 성공	민서 CSV replay 로 risk 0-3 출력, 진동/음성 구분, v2x_uart_log.csv 확보	Jetson 자동 부팅, ONNX 추론, 완전 매립
대성공	CSV replay 모드와 실제 ESP32 송신 모드 모두 성공, 1 분 이상 E2E 영상, GitHub final 정리	PCB/만능기판, 제품형 케이스

2. 현장 전체 운영 방식과 시간표

6 시간을 기준으로 작성하지만, 핵심은 시간이 아니라 게이트 통과이다. 각 시간대의 목표를 못 끝냈으면 다음 시간대로 억지로 넘어가지 말고, 최소 성공 루트로 전환한다.

시간	공통 목표	박중선	강현준	최민서	게이트
00:00-00:20	자리 배치, 전원 분리, 코드 백업	하드웨어 전체를 책상 위에 펼치고 송신기/수신기/출력부 라벨 확인	Jetson/노트북 부팅, 프로젝트 폴더 생성	CSV 원본, Python 환경, 그래프 저장 폴더 준비	모든 장비 전원 켜기 전 사진 1 장
00:20-00:50	전원과 포트부터 확인	ESP32 2 개 COM 포트 확인, 모터/DFPlayer 전원 분리 확인	Jetson 전원: 어댑터 또는 배터리 방식 확인, Python/포트 확인	pandas/matplotlib 설치 확인	보드가 켜지고 포트가 보임
00:50-01:30	4 차 기능 재현	송신기 [SEND], 수신기 [RX], risk 0-3 출력 재현	Serial Monitor 로그 확인, 파일명 working 복사	fcd_trajectory.csv 컬럼 확인	4 차 기능이 다시 됨
01:30-02:20	Jetson 전원/배터리/통신 1 차	Jetson 배터리 연결 보조, J48/배럴잭/전압 확인	Jetson listener 실행, UART 또는 USB Serial 로 수신	보행자 기준점과 시나리오 3 개 설계	Jetson 에 한 줄 이상 데이터
02:20-03:20	CSV replay 시연 만들기	출력부가 RISK 명령을 받게 준비, 진동/음성 확인	risk_replay_from_csv.py 실행	risk_labeled_fcd.csv, scenario CSV 생성	risk 가 시간에 따라 변환
03:20-04:20	실제 ESP32 송신 모드	송신기 흔들기, GPS/IMU 값 변화 확인, 선 안정화	ESP-NOW -> Jetson -> CSV 저장	risk 지도/타임라인 png 생성	실제 하드웨어 로그 확보
04:20-05:10	외관 정리와 전원 안정화	고정판/박스/라벨링, 케이블 타이, 모터/스피커 위치 정리	최종 코드 경로 정리, GitHub commit 준비	산출물 폴더명 정리, 설명문 작성	다시 켜도 동작
05:10-06:00	최종 촬영과 백업	출력 장치 클로즈업 담당	터미널/CSV/GitHub 담당	그래프와 시나리오 설명 담당	영상/사진/CSV/README 확보

2.1 자리 배치

- 책상 왼쪽: 박중선 하드웨어 존. 송신기, 수신기, DFPlayer, 진동모터, 스피커, 배터리, 멀티미터를 둔다.
- 책상 중앙: 강현준 연결성 존. Jetson Nano, 노트북, USB Serial, UART 점퍼선, CSV 로그 화면을 둔다.
- 책상 오른쪽: 최민서 데이터 존. SUMO/CSV, risk 그래프, scenario CSV 생성 화면을 둔다.
- 모든 보드의 GND/5V/3V3 라벨이 보이게 놓는다. 전원선을 연결하기 전 반드시 사진을 찍어 원상복구 기준을 만든다.
- 처음부터 박스에 넣지 않는다. 먼저 동작 확인, 그 다음 고정, 마지막에 다시 동작 확인이다.

3. 팀원별 역할 총괄표 - 누가 무엇을 끝내야 하는가

팀원	이번 회차 이름	핵심 책임	본인이 직접 해야 하는 것	남겨야 하는 증거
박중선	하드웨어 재현/전원 안정성/외관 정리 담당	4 차에서 된 하드웨어를 오늘도 되게 하고, 전원 문제로 꺼지지 않게 하며, 발표 가능한 형태로 정리한다.	송신기/수신기 재현, risk 출력부 확인, Jetson 배터리 연결 보조, 배선 라벨링, 코드/핀맵 정리	risk 출력 영상, 전원 연결 사진, Jetson 배터리 연결 사진, 핀맵표, 외관 정리 사진
강현준	전체 연결성/Jetson/CSV/GitHub 담당	송신기-수신기-Jetson-CSV-risk 출력의 전체 길을 만든다.	Jetson listener, UART/USB Serial, CSV 저장, risk replay, GitHub final commit	터미널 캡처, v2x_uart_log.csv, risk replay 영상, GitHub 캡처
최민서	위험 시나리오/risk 라벨/시각화 담당	차량 궤적을 위험도 0-3 으로 바꾸고, 왜 지팡이가 울리는지 데이터로 설명한다.	scenario CSV 3 개, distance/TTC/risk 계산, PNG 3 개, risk_label_rule 작성	risk_labeled_fcd.csv, scenario_*.csv, risk_colored_map.png, risk_timeline.png
전원	최종 시연/검증 담당	각자 성공한 것을 하나의 영상과 README 로 합친다.	1 분 이상 통합 영상, 실패/해결 기록, 다음 회차 작업 도출	e2e_demo.mp4, README_result.txt, 사진/영상/코드 백업

4. 박중선 파트 - 하드웨어 재현, Jetson 배터리 연결, 출력부, 외관 정리

박중선 파트는 이번 수정본에서 가장 자세하게 늘린다. 이유는 단순히 “모터/DFPlayer 가 된다”가 아니라, 하드웨어가 실제 통합 환경에서 안정적으로 작동해야 5 차가 성공하기 때문이다. 박중선은 하드웨어 전원 안정성 담당으로 Jetson 배터리 연결까지 같이 확인한다.

4.1 박중선이 시작 전에 알아야 할 원칙

★전원선은 가장 위험하다. 빨간색이라고 무조건 5V, 검은색이라고 무조건 GND 라고 믿지 말고 멀티미터 또는 보드 라벨로 확인한다.

- ESP32 의 3V3 핀은 센서 전원용이지 모터 전원용이 아니다. 진동모터나 DFPlayer 를 3V3 에 무리하게 물리지 않는다.
- 모터를 ESP32 GPIO 에 직결하지 않는다. 트랜지스터/MOSFET/드라이버를 거쳐야 한다.
- DFPlayer Mini 는 순간 전류를 먹기 때문에 ESP32 가 리셋되면 전원 분리와 캐패시터를 먼저 의심한다.
- Jetson Nano 는 5V 가 맞지만 전류가 부족하면 꺼진다. “전압 5V”만 맞다고 끝이 아니라, “충분한 전류”가 필요하다.
- Jetson 의 5V GPIO 핀에 배터리를 직접 넣는 방식은 하지 않는다. 보호회로를 우회할 수 있으므로 DC Barrel Jack 을 우선 사용한다.
- 이번 회차에서는 만능기판 납땜을 하지 않는다. 연결 검증이 끝나기 전 납땜하면 나중에 디버깅이 어려워진다.

4.2 박중선 작업 순서 1 부터 10 까지

1. 하드웨어를 전부 책상에 펼치고 송신기, 수신기, 출력부, Jetson, 배터리를 분리해서 놓는다.
2. 각 보드에 라벨을 붙인다. “송신기 ESP32”, “수신기 ESP32”, “Jetson”, “Motor/DFPlayer 전원”, “GPS/IMU”처럼 적는다.
3. 전원을 켜기 전에 위에서 전체 사진 1 장을 찍는다. 이 사진은 나중에 선이 빠졌을 때 복구 기준이 된다.
4. 송신기 ESP32 만 노트북에 꽂아 4 차 working sender 코드가 아직 동작하는지 확인한다.
5. 수신기 ESP32 만 노트북에 꽂아 4 차 working receiver 코드가 아직 동작하는지 확인한다.
6. 송신기와 수신기를 동시에 켜서 ESP-NOW [SEND]/[RX]가 증가하는지 확인한다.
7. risk 강제 테스트 또는 Jetson RISK 명령으로 risk 0/1/2/3 진동/음성을 확인한다.
8. Jetson 전원을 어댑터 또는 배터리로 켜고, J48 점퍼/배럴잭/전원 LED/부팅 안정성을 확인한다.
9. 전체 연결이 성공한 뒤에만 고정판/박스/라벨링을 시작한다.
10. 최종적으로 선을 조금 흔들어도 꺼지지 않는지 확인하고, 사진/영상/핀맵/전원 구조를 README 에 적는다.

4.3 박중선 세부 작업 A - 송신기 재현

단계	박중선이 직접 하는 행동	성공 기준	실패하면
A-1	송신기 ESP32 를 노트북 USB 에 연결한다. Arduino IDE 에서 보드와 COM 포트를 선택한다.	장치 관리자 또는 Arduino IDE 에 COM 포트가 보인다.	케이블을 데이터 케이블로 교체하고 CP210x/CH340 드라이버 확인
A-2	v2x_sender_20260517_working.ino 를 연다. 수신기 MAC 주소가 맞는 지 확인한다.	코드가 컴파일된다.	MAC 주소가 틀리면 수신기 Serial 에서 출력되는 MAC 으로 수정
A-3	업로드 후 Serial Monitor 를 115200 baud 로 연다.	[SEND] seq 가 0,1,2... 증가한다.	보드 리셋 버튼, 포트 재선택, Upload Speed 낮추기
A-4	IMU 를 손으로 기울이고 흔든다.	ax/ay/az 또는 gx/gy/gz 값이 변한다.	I2C SDA/SCL, 센서 전원, 라이브러리

			초기화 확인
A-5	GPS 는 실내에서 valid=0 이어도 일단 실패로 보지 않는다.	GPS Serial 이 시작되고 chars 또는 raw 값이 들어온다.	야외에서 3-10 분 cold start, 안테나 방향 확인

4.4 박종선 세부 작업 B - 수신기와 출력부 재현

단계	행동	성공 기준	주의
B-1	수신기 ESP32 에 receiver working 코드를 업로드한다.	Serial Monitor 에 Receiver ready 또는 MAC 출력	송신기와 수신기 코드를 서로 바꾸지 않기
B-2	송신기를 1m 안쪽에서 켜고 수신기 로 그를 본다.	[RX] seq 가 증가	MAC 주소, 구조체 크기, 채널 확인
B-3	risk 강제 테스트 코드를 켜서 risk 를 5 초마다 0-1-2-3 으로 바꾼다.	모터와 음성 출력이 단계별로 다름	테스트 코드가 final 에 섞이지 않게 파일명 분리
B-4	진동모터 작동 순간 ESP32 로그가 끊기는지 본다.	30 초 이상 리셋 없음	브라운아웃이면 모터 전원 분리, GND 공통, 캐패시터 추가
B-5	DFPlayer 음성을 확인한다.	0001.mp3, 0002.mp3 등이 재생됨	SD 카드 폴더명, 파일명, 스피커 극성, 볼륨 확인

4.5 박종선 세부 작업 C - Jetson 배터리 연결 추가 파트

이번 수정본에서 추가된 핵심이다. Jetson 은 프로젝트의 엣지 컴퓨팅 노드 또는 RSU 역할이다. 콘센트만으로 시연하면 야외 보행 시연 확장성이 약해 보이므로, 배터리로도 켤 수 있는지 확인한다. 단, 안전이 최우선이다.

준비물	필요 이유	확인 방법
Jetson Nano Developer Kit	중앙 연산/로그 저장/위험도 계산 노드	전원 LED 와 부팅 화면 확인
5V 4A 어댑터 또는 안정적인 5V 4A 급 배터리 구성	Jetson 은 5V 전압뿐 아니라 충분한 전류가 필요	부팅 후 부하가 걸려도 꺼지지 않음
USB-PD 지원 보조배터리	야외/이동 전원 공급	PD 출력 지원 표기 확인
USB-C to DC 5V 트리거 케이블 또는 5V 고전류 DC 케이블	보조배터리의 5V 를 DC Barrel Jack 으로 전달	출력 전압이 5V 인지 확인
5.5mm x 2.1mm DC Barrel Jack 규격 케이블	Jetson DC 포트 연결	헐겁지 않고 끝까지 삽입됨
J48 점퍼 캡	Jetson 이 DC Barrel Jack 전원을 쓰도록 선택	J48 핀 두 개가 점퍼로 연결됨
멀티미터	전압 극성/5V 확인	센터 플러스, 5V 근처 확인

4.5.1 Jetson 배터리 연결 절차 - 반드시 순서대로

1. Jetson 전원을 모두 분리한다. Micro-USB, DC Jack, USB 주변장치를 모두 뺀다.
2. Jetson 보드에서 J48 Power Select Header 위치를 확인한다. J48 은 DC Barrel Jack 전원 사용을 선택하는 점퍼다.
3. J48 핀 두 개에 점퍼 캡을 꽂는다. 점퍼가 없으면 DC Barrel Jack 에 전원을 넣어도 켜지지 않을 수 있다.
4. 보조배터리가 USB-PD 출력 가능한지 확인한다. 가능하면 5V 에서 3A 이상, 안정적으로 4A 급을 권장한다.
5. USB-C to DC 5V 트리거 케이블을 보조배터리에 연결한다. 9V/12V 트리거 케이블이 아니라 반드시 5V 출력인지 확인한다.
6. 멀티미터로 DC 플러그의 극성을 확인한다. 일반적으로 중앙이 +, 바깥이 - 이지만, 케이블마다 확인해야 한다.
7. 멀티미터로 전압이 5V 근처인지 확인한다. 9V 나 12V 가 나오면 절대 Jetson 에 꽂지 않는다.
8. Jetson DC Barrel Jack 에 플러그를 꽂는다. 전원 LED 가 켜지는지 확인한다.
9. 부팅이 완료되면 터미널에서 `sudo nvpmode -q` 로 전력 모드를 확인한다.
10. 고성능이 필요하면 `sudo nvpmode -m 0` 을 입력한다. 이후 간단한 Python script 와 CSV 저장을 실행해도 꺼지지 않는지 확인한다.
11. 배터리 연결 사진을 반드시 찍는다. J48 점퍼, 보조배터리, DC 케이블, Jetson 전원 LED 가 한 화면에 보이면 좋다.
12. 5 분 이상 켜둔 상태에서 Jetson 이 재부팅하지 않으면 “Jetson 배터리 전원 1 차 성공”으로 기록한다.

절대 금지: Jetson 40 핀 헤더의 5V/GND 에 보조배터리 전원을 직접 넣지 않는다. 보호회로를 우회할 수 있고, 극성 실수나 과전류로 보드가 손상될 수 있다. 이번 회차는 DC Barrel Jack + J48 점퍼 방식을 우선한다.

4.5.2 Jetson 전원 방식 판정표

전원 방식	이번 회차 사용 여부	이유	판정
Micro-USB 5V 2A	가급적 사용하지 않음	가벼운 부팅은 가능해도 부하 시 전압 강하/재부팅 가능성이 큼	임시 확인용만
DC Barrel Jack 5V 4A 어댑터	권장	전류 여유가 있고 가장 안정적	실내 통합 테스트 기본

USB-PD 보조배터리 + 5V DC 트리거 케이블	권장/추가 목표	야외 시연과 이동성 확보	박중선이 연결 확인
GPIO 5V 핀 직접 주입	금지	보호회로 우회 위험	하지 않음
18650 직렬팩 직접 연결	금지 또는 별도 BMS/강압회로 필요	전압이 5V 가 아니며 과방전/과전류 위험	이번 회차에서 직접 연결 금지

4.5.3 Jetson 배터리 연결 성공 기준

체크	완료 조건	증거
☐ J48 점퍼	J48 에 점퍼 캡이 꽂혀 있음	클로즈업 사진
☐ 케이블 전압	DC 플러그 출력이 5V 근처임	멀티미터 사진 또는 기록
☐ 극성	중앙 + / 바깥 - 확인	기록
☐ 부팅	전원 LED 와 Ubuntu 부팅 확인	사진/영상
☐ 안정성	5 분 이상 켜져 있고 터미널 사용 중 재부팅 없음	영상 또는 시간 기록
☐ 통합 부하	listener 또는 risk replay 실행 중 꺼지지 않음	터미널 캡처

4.6 박중선 핀맵 기록 양식

보드	핀/GPIO	연결 대상	역할	전압/주의
송신기 ESP32	3V3	IMU VCC, GPS VCC(모듈 사양 확인)	센서 전원	GPS 모듈이 5V 허용인지 3.3V 전용인지 확인
송신기 ESP32	GND	IMU/GPS 공통	기준 접지	모든 센서와 공통 GND
송신기 ESP32	GPIO21	IMU SDA	I2C 데이터	코드와 실제 핀 일치 확인
송신기 ESP32	GPIO22	IMU SCL	I2C 클럭	SDA/SCL 바뀌지 않게 라벨
송신기 ESP32	GPIO16 RX2	GPS TX	GPS 데이터 수신	TX -> RX 교차
송신기 ESP32	GPIO17 TX2	GPS RX 또는 옵션	GPS 설정 송신	사용하지 않으면 연결 생략 가능
수신기 ESP32	GPIO17 TX2	Jetson RX 또는 USB Serial RX	JSON/RISK 데이터 송신	Jetson Pin10 RXD 에 연결할 때 GND 공유
수신기 ESP32	GPIO16 RX2	Jetson TX 또는 USB Serial TX	Jetson 명령 수신	RISK 명령 받을 때 사용
수신기 ESP32	GPIO26 예시	모터 드라이버 입력	진동 출력	모터 직결 금지, 실제 코드 핀 우선
수신기 ESP32	GPIO25/32 예시	DFPlayer RX/TX	음성 제어	DFPlayer RX 에 1k 저항 권장

4.7 박중선 외관 정리 기준

- 외관 정리는 “완제품처럼 숨기기”가 아니라 “발표 가능한 실험용 시제품처럼 정리하기”이다.
- 디버깅이 끝나기 전에는 글루건을 과하게 쓰지 않는다. 선을 다시 뽑을 수 있어야 한다.
- ESP32 USB 포트는 밖으로 향하게 둔다. 나중에 코드 업로드가 가능해야 한다.
- GPS 안테나는 박스 안 깊숙이 넣지 않는다. 하늘 방향으로 빼거나 위쪽에 둔다.
- 진동모터는 사용자가 손으로 잡는 쪽 또는 지팡이 손잡이 근처에 둔다.
- 스피커는 막히지 않게 구멍 또는 바깥 방향으로 둔다.
- 전원선과 신호선을 섞어 묶지 말고, 라벨을 붙인다: 5V, 3V3, GND, TX, RX, Motor, Speaker, GPS, IMU.

4.8 박중선 현장 작업 카드 - 그대로 따라 하는 버전

아래 카드는 박중선이 현장에서 순서대로 체크하면서 실행하는 카드이다. “내가 지금 뭘 해야 하지?”가 헛갈리면 이 부록만 보고 움직인다. 각 카드 끝에는 반드시 사진, 영상, 파일명, 성공/실패 기록 중 하나를 남긴다.

A-1. 작업 시작 전 하드웨어 분류 카드

1. 책상 위에 송신기 ESP32, 수신기 ESP32, Jetson Nano, 진동모터, DFPlayer, 스피커, GPS, IMU, 보조배터리, USB 케이블을 모두 꺼낸다.
2. 송신기와 수신기 ESP32 가 바뀌지 않게 종이 라벨을 붙인다. 보드가 바뀌면 MAC 주소와 코드가 꼬인다.
3. 각 보드 위에 “전원 입력 위치”, “GND”, “TX/RX”, “센서 연결부”를 포스트잇으로 표시한다.

4. 전원을 켜기 전 전체 사진을 찍는다. 이 사진은 선이 빠졌을 때 복원 기준이다.
5. 노트북에 연결할 USB 케이블은 데이터 전송이 되는 케이블인지 확인한다. 충전 전용 케이블은 업로드가 안 된다.
6. 멀티미터가 있으면 5V/GND 극성 확인용으로 바로 꺼내 둔다.

A-2. 송신기 ESP32 재현 카드

1. 송신기 ESP32 만 노트북에 연결한다. 다른 보드는 아직 켜지 않는다.
2. Arduino IDE 에서 보드를 ESP32 Dev Module 로 선택하고, 새로 생긴 COM 포트를 선택한다.
3. v2x_sender_20260517_working.ino 를 연다. 파일명에 working 이 없으면 먼저 복사본을 만든다.
4. 업로드 전 코드 상단의 receiverAddress 또는 peer MAC 주소가 4 차 성공 수신기 MAC 인지 확인한다.
5. 업로드 후 Serial Monitor 115200 baud 를 연다.
6. [SEND] seq 가 증가하면 성공이다. seq 가 증가하지 않으면 업로드 실패, 코드 멈춤, 센서 초기화 실패를 순서대로 본다.
7. IMU 가 있다면 보드를 기울여 ax/ay/az 값이 변하는지 확인한다. 값이 고정이면 I2C 배선을 본다.
8. GPS 가 있다면 실내에서는 valid=0 이어도 정상일 수 있다. 대신 GPS serial begin 메시지나 raw 문자가 들어오는지만 확인한다.
9. 성공 화면을 캡처하고 sender working 파일을 백업 폴더에 복사한다.

A-3. 수신기 ESP32 재현 카드

1. 수신기 ESP32 만 노트북에 연결한다. 송신기와 포트가 헛갈리면 장치 관리자에서 뺐다 꽂아 확인한다.
2. v2x_receiver_20260517_working.ino 를 열고 업로드한다.
3. Serial Monitor 115200 baud 에서 수신기 MAC 주소와 Receiver ready 문구가 나오는지 확인한다.
4. 송신기를 켜고 두 보드를 1m 이내에 둔다.
5. 수신기 Serial Monitor 에 [RX] seq 가 증가하면 ESP-NOW 1 차 성공이다.
6. [RX]가 안 뜨면 송신기 MAC 주소, esp_now_register_rcv_cb, 구조체 크기, WiFi 채널을 확인한다.
7. 성공하면 30 초 정도 영상을 찍는다. 영상에는 송신기와 수신기 로그가 동시에 보이면 좋다.

A-4. risk 출력부 재현 카드

1. 진동모터와 DFPlayer 가 연결된 수신기/출력부를 준비한다.
2. risk 강제 테스트 코드를 사용해 5 초마다 risk 가 0-1-2-3 으로 바뀌게 한다.
3. risk 0에서는 진동이 꺼지거나 매우 약해야 한다. risk 1 은 약하게, risk 2 는 중간, risk 3 은 강하게 구분한다.
4. 음성 파일은 /mp3/0001.mp3, /mp3/0002.mp3, /mp3/0003.mp3 형식인지 확인한다.
5. DFPlayer 가 무음이면 볼륨 코드, 스피커 연결, TX/RX 교차, 5V 전원을 순서대로 확인한다.
6. 진동모터가 켜지는 순간 ESP32 가 재부팅되면 출력부 전원이 부족한 것이다. 모터 전원을 ESP32 3V3 에서 빼고 별도 5V 로 분리한다.
7. 30 초 이상 risk 0-3 반복 중 ESP32 가 리셋되지 않으면 출력부 안정성 1 차 통과다.

A-5. Jetson 배터리 연결 카드 - 박종선 추가 담당

1. Jetson 을 만지기 전에 손에 들고 있는 케이블이 5V 출력인지 확인한다. 9V/12V 트리거 케이블이면 절대 사용하지 않는다.
2. Jetson Nano 보드의 J48 Power Select Header 위치를 찾고 점퍼 캡을 꽂는다.
3. 보조배터리 USB-C 포트에 5V DC 트리거 케이블을 연결한다.
4. DC 플러그를 Jetson 에 꽂기 전 멀티미터로 전압을 확인한다. 5V 근처가 아니면 중단한다.
5. 극성은 중앙 +, 바깥 -인지 확인한다. 반대면 중단한다.
6. DC Barrel Jack 에 꽂고 Jetson 전원 LED 가 켜지는지 확인한다.
7. 부팅 후 터미널에서 5 분 이상 유지되는지 본다. 중간에 꺼지면 전류 부족 또는 케이블 전압강하 가능성이 크다.
8. J48 점퍼, 보조배터리, DC 케이블, Jetson LED 가 모두 보이게 사진을 찍는다.
9. 성공하면 README_result 에 “Jetson 배터리 부팅 성공/실패, 사용 배터리, 사용 케이블, 유지 시간”을 적는다.

A-6. 전원 안정화 카드

1. 송신기, 수신기, 출력부, Jetson 전원을 한 줄로 모아놓지 않는다. 각각 분리해서 생각한다.
2. ESP32 센서 전원과 모터/스피커 전원은 가능하면 분리한다. 단, 신호 기준을 맞추기 위해 GND 는 공통으로 묶는다.
3. 모터 근처에 100uF 이상 캐패시터가 있으면 추가한다. 순간 전류 변동을 줄인다.
4. 모터 드라이버 또는 트랜지스터 회로에서 다이오드 방향을 확인한다. 코일성 부하는 역기전력 보호가 필요하다.
5. USB 케이블이 흔들려서 전원이 끊기지 않도록 케이블타이로 고정판에 묶는다.
6. 전원 안정성 테스트는 “켜짐”이 아니라 “움직여도 안 꺼짐”이다. 선을 약하게 흔들어도 로그가 유지되어야 한다.

A-7. 외관 정리 카드

1. 전체 기능이 먼저 성공한 뒤 외관 정리를 시작한다.
2. 브레드보드를 폼보드나 아크릴판 위에 붙인다. USB 포트는 바깥을 향하게 둔다.
3. GPS 안테나는 박스 밖이나 위쪽으로 빼둔다. 박스 내부에 깊이 넣으면 수신이 더 안 될 수 있다.

- 스피커는 구멍이 있는 방향 또는 바깥쪽을 향하게 둔다.
- 진동모터는 손잡이 쪽에 최대한 가깝게 둔다. 책상 위에서만 진동하면 발표 때 체감이 약하다.
- 전원선, GND, TX/RX, Motor, Speaker, GPS, IMU 라벨을 붙인다.
- 외관 정리 후 다시 전원을 켜서 [SEND], [RX], Jetson CSV, risk 출력이 그대로 되는지 확인한다.

A-8. 박중선 코드/핀맵 정리 카드

- 최종 영상에 사용한 sender 코드를 v2x_sender_20260517_final.ino 로 저장한다.
- 최종 영상에 사용한 receiver 코드를 v2x_receiver_20260517_final.ino 로 저장한다.
- risk 강제 테스트가 들어간 코드는 final 에 섞지 말고 test_risk_force 이름으로 분리한다.
- 핀맵표에는 ESP32 핀 번호, 연결 대상, 전원/신호 구분, 방향, 주의사항을 적는다.
- 라이브러리 이름을 기록한다. 예: TinyGPSPlus, DFRobotDFPlayerMini, Wire, esp_now 등.
- 배선 사진은 송신기, 수신기/출력부, Jetson 배터리, 전체 배치로 나누어 찍는다.

4.9 선 연결을 어디까지 자세히 알려줘야 하는가

팀원이 헛갈릴 수 있는 부분은 “선을 몇 번에 꽂아라” 수준으로 알려주는 것이 맞다. 다만 보드마다 핀 위치가 다르고, ESP32 DevKit 도 제조사에 따라 실크 인쇄가 다를 수 있으므로, 문서에는 핀 번호와 함께 “보드 라벨을 확인하라”는 절차를 반드시 같이 적어야 한다.

상황	얼마나 자세히 적어야 하나	예시
전원 연결	가장 자세히. 전압, 극성, 금지 연결까지 적어야 함	Jetson 은 J48 점퍼 후 DC Barrel Jack 5V 4A. GPIO 5V 직접 주입 금지.
UART 연결	TX/RX 교차와 GND 공유까지 적어야 함	ESP32 TX2 GPIO17 -> Jetson Pin10 RXD, ESP32 GND -> Jetson Pin6 GND
I2C 센서	기본 핀과 코드 핀을 같이 적어야 함	ESP32 GPIO21 SDA, GPIO22 SCL. 단, 코드에서 Wire.begin(SDA,SCL) 확인
모터 출력	직결 금지와 드라이버 사용을 적어야 함	GPIO -> 저항/트랜지스터 게이트 -> 모터 전원. 모터를 GPIO 에 직접 연결하지 않음
DFPlayer	전원, TX/RX, SD 파일명까지 적어야 함	DFPlayer VCC 5V, GND 공통, RX/TX 교차, /mp3/0001.mp3 형식
GPS	실내 valid=0 정상 가능성을 적어야 함	GPS TX -> ESP32 RX2, 야외 cold start 3-10 분

5. 강현준 파트 - 전체 연결성, Jetson 수신, CSV 저장, risk replay, GitHub

강현준 파트는 “전체가 연결되는지”를 보는 파트다. 박중선이 하드웨어를 안정화하고, 최민서가 위험 CSV 를 만들면, 강현준은 이것을 Jetson 과 코드로 묶어야 한다. 실제 송신기 모드와 민서 CSV replay 모드를 모두 준비한다.

5.1 강현준이 당일 해야 하는 일 1 부터 10 까지

- 프로젝트 폴더를 생성하고 working/test/final 구조를 만든다.
- Jetson 을 안정적인 전원으로 부팅한다. 가능하면 박중선과 함께 배터리 부팅도 확인한다.
- Python3, pip, pyserial, pandas 설치 여부를 확인한다.
- 헤더 UART 를 쓰는 경우 nvgetty 를 중지하고 재부팅한다.
- 수신기 ESP32 에서 Jetson 으로 JSON 한 줄이 들어오는지 확인한다.
- jetson_listener.py 를 실행하여 seq/risk/lat/lng 를 터미널에 출력한다.
- v2x_uart_log.csv 줄 수가 증가하는지 wc -l 또는 tail -f 로 확인한다.
- 최민서가 만든 scenario CSV 를 risk_replay_from_csv.py 로 읽어서 ESP32 출력부에 RISK 명령을 보낸다.
- 실제 송신기 모드와 CSV replay 모드를 각각 영상으로 남긴다.
- 최종 코드와 README_result.txt 를 GitHub 에 정리한다.

5.2 Jetson UART/USB Serial 선택 기준

방식	장점	단점	이번 회차 판정
헤더 UART /dev/ttyTHS1 또는 /dev/ttyS0	실제 임베디드 연결에 가깝고 발표 설득력이 좋음	TX/RX/GND, nvgetty, 권한 문제가 생길 수 있음	우선 시도
USB Serial /dev/ttyUSB0 또는 /dev/ttyACM0	가장 안정적이고 빠르게 성공 가능	헤더 UART 실증은 다음 회차로 밀릴 수 있음	헤더 UART 가 막히면 즉시 대체
PC Serial 대체	Jetson 문제와 ESP32 문제를 분리 가능	Jetson 실기 검증은 아님	최후의 디버깅용

5.3 Jetson 헤더 UART 기본 연결

ESP32 수신기	Jetson Nano 40 핀	의미	주의
ESP32 TX2 / GPIO17	Jetson Pin 10 RXD	ESP32 가 보내는 데이터를 Jetson 이 받음	TX -> RX 교차
ESP32 RX2 / GPIO16	Jetson Pin 8 TXD	Jetson 이 보내는 RISK 명령을 ESP32 가 받음	RX -> TX 교차, 명령 받을 때만 필요
ESP32 GND	Jetson Pin 6 GND	기준 전압 공유	공통 GND 없으면 글자 깨짐
ESP32 5V/3V3	Jetson 5V/3V3	연결하지 않음	각자 전원 사용, 신호선과 GND 만 연결

5.4 Jetson 터미널 명령 순서

```
# Python 확인
python3 --version
python3 -m pip --version
# 필요한 패키지 설치
python3 -m pip install pyserial pandas
# 시리얼 포트 후보 확인
ls -l /dev/ttyTHS1
ls -l /dev/ttyS0
ls -l /dev/ttyUSB0
ls -l /dev/ttyACM0
# 헤더 UART 사용 시 nvgetty 중지
sudo systemctl stop nvgetty
sudo systemctl disable nvgetty
sudo udevadm trigger
sudo usermod -aG dialout $USER
sudo reboot
# 재부팅 후 로그 수신
cd ~/v2x/03_jetson
python3 jetson_listener_20260517_final.py
# 다른 터미널에서 CSV 증가 확인
tail -f v2x_uart_log.csv
wc -l v2x_uart_log.csv
```

5.5 강현준 완료 기준

체크	완료 조건	증거
☐ Jetson 부팅	어댑터 또는 배터리로 안정 부팅	사진
☐ 포트 확인	/dev/ttyTHS1 또는 /dev/ttyUSB0 등 실제 포트 기록	터미널 캡처
☐ 데이터 수신	seq/risk 가 터미널에 출력	영상/캡처
☐ CSV 저장	v2x_uart_log.csv 줄 수 증가	wc -l 캡처
☐ CSV replay	scenario_front_approach.csv 로 RISK 0-3 전송	터미널 캡처
☐ 출력 연동	ESP32 출력부에서 진동/음성 반응	영상
☐ GitHub 정리	final 코드와 README 업로드	GitHub Desktop 캡처

5.6 강현준 현장 작업 카드 - 연결성 담당 세부 절차

B-1. Jetson 부팅 및 환경 체크 카드

1. Jetson 전원은 5V 4A 어댑터를 기본으로 하고, 박종선이 준비한 배터리 방식도 별도로 확인한다.
2. 부팅 후 터미널을 열고 python3 --version 을 입력한다.
3. python3 -m pip --version 을 입력한다. pip 가 없으면 설치부터 해결한다.

- python3 -m pip install pyserial pandas 를 실행한다.
- ls -l /dev/ttyTHS1, /dev/ttyS0, /dev/ttyUSB0, /dev/ttyACM0 로 포트 후보를 본다.
- 실제 사용한 포트를 README_result 에 적는다. 나중에 포트가 달라지면 디버깅 시간이 늘어난다.

B-2. 헤더 UART 사용 카드

- ESP32 TX2(GPIO17)를 Jetson RXD(Pin10)에 연결한다.
- ESP32 RX2(GPIO16)를 Jetson TXD(Pin8)에 연결한다. Jetson 에서 명령을 보내지 않는다면 일단 생략 가능하다.
- ESP32 GND 와 Jetson GND(Pin6)를 반드시 연결한다.
- ESP32 5V/3V3 와 Jetson 5V/3V3 는 연결하지 않는다.
- nvgetty 가 UART 를 잡고 있으면 stop/disable 후 재부팅한다.
- jetson_listener 를 실행했을 때 아무것도 안 나오면 TX/RX 를 다시 확인한다.

B-3. USB Serial 대체 카드

- 헤더 UART 가 20 분 이상 막히면 USB Serial 로 전환한다.
- 수신기 ESP32 를 USB 로 Jetson 에 연결한다.
- Jetson 에서 ls /dev/ttyUSB0 또는 /dev/ttyACM0 가 생기는지 확인한다.
- jetson_listener.py 실행 시 포트를 /dev/ttyUSB0 로 넘긴다.
- CSV 저장이 되면 5 차 최소 목표는 성공으로 처리한다.
- 헤더 UART 실패 이유는 README 에 따로 적고, 6 차 첫 작업으로 넘긴다.

B-4. CSV 저장 검증 카드

- jetson_listener 실행 후 터미널에 seq 가 찍히는지 본다.
- 다른 터미널에서 tail -f v2x_uart_log.csv 를 실행한다.
- 30 초 간격으로 wc -l v2x_uart_log.csv 를 두 번 입력한다.
- 줄 수가 증가하면 CSV 저장 성공이다.
- CSV 가 생성되지 않으면 현재 폴더와 파일 권한을 확인한다.
- CSV 는 최종 폴더에 v2x_uart_log_20260517.csv 이름으로 복사한다.

B-5. 민서 CSV replay 연결 카드

- 최민서에게 scenario_front_approach.csv 를 받는다.
- CSV 컬럼에 time, risk_level, distance_to_ped, ttc 가 있는지 확인한다.
- ESP32 출력부를 Jetson 명령 수신 코드로 준비한다.
- python3 risk_replay_from_csv.py scenario_front_approach.csv /dev/ttyUSB0 를 실행한다.
- 터미널에 RISK,0 부터 RISK,3 까지 흐르는지 확인한다.
- 박중선에게 각 risk 마다 진동/음성이 맞는지 확인하게 한다.
- side_blindspot 과 safe_pass 도 실행한다. safe_pass 에서 과하게 올리면 민서 risk 기준을 조정한다.

B-6. GitHub 정리 카드

- GitHub Desktop 에서 팀 저장소를 clone 한다. 저장 위치는 OneDrive 가 아닌 C:\Projects 같은 일반 로컬 폴더를 권장한다.
- 작업 전 Pull origin 을 눌러 최신 상태를 받는다.
- 카톡으로 받은 파일명을 그대로 올리지 말고 01_sender, 02_receiver, 03_jetson, 04_minseo 구조에 넣는다.
- Changes 화면에서 올라가는 파일을 확인한다. 임시 캐시, pycache, 불필요한 큰 영상은 제외한다.
- 커밋 메시지는 “5th meeting final e2e connection”처럼 실제 내용이 보이게 적는다.
- Commit 후 Push origin 을 누르고, 성공 화면을 캡처한다.

6. 최민서 파트 - SUMO/CSV 위험 시나리오, risk 계산, 시각화

최민서 파트는 단순히 Transformer 코드를 실행하는 역할이 아니다. 5 차에서는 “왜 위험도가 올라갔는지”를 데이터로 설명하는 파트가 핵심이다. 실제 차량 실험은 위험하고 재현성이 낮기 때문에, SUMO/CSV 기반 시나리오를 만들어 Jetson 에서 반복 재생할 수 있게 한다.

6.1 최민서가 당일 해야 하는 일 1 부터 10 까지

- fcd_trajectory.csv 를 04_minseo_sumo_ai 폴더에 넣는다.
- CSV 컬럼을 확인한다. time, vehicle_id, x, y, speed 가 있는지 본다.
- 보행자 기준점 ped_x, ped_y 를 정한다. 처음에는 고정점으로 한다.
- 차량과 보행자 사이 거리 distance_to_ped 를 계산한다.
- 거리 변화량으로 relative_speed 를 계산한다. 거리가 줄어드는 방향이면 접근 중으로 본다.
- TTC(Time To Collision)를 계산한다. 접근하지 않으면 999 같은 큰 값으로 둔다.
- distance 와 TTC 기준으로 risk_level 0/1/2/3 을 붙인다.

- 정면 접근, 측면/사각지대 접근, 비위험 통과 CSV 3 개를 만든다.
- trajectory_map.png, risk_colored_map.png, risk_timeline.png 를 만든다.
- 강현준에게 scenario_front_approach.csv 를 먼저 전달하고 risk replay 연결을 확인한다.

6.2 risk 레벨 기준

조건	risk_level	출력 의미	발표 설명
멀거나 접근하지 않음	0	무음 또는 매우 약한 상태	오경보를 줄이기 위해 무조건 울리지 않는다.
distance < 10m 또는 TTC < 5 초	1	주의 진동/짧은 안내	접근 가능성이 생겨 사용자에게 약하게 알린다.
distance < 6m 또는 TTC < 3 초	2	중간 진동/경고 음성	위험 가능성이 높아져 명확하게 경고한다.
distance < 3m 또는 TTC < 1.5 초	3	강한 진동/즉시 정지 안내	충돌 가능성이 높아 즉시 정지 수준으로 안내한다.

6.3 최민서 산출물

파일	내용	강현준/박중선과 연결되는 부분
risk_labeled_fcd.csv	원본 FCD 에 distance, relative_speed, ttc, risk_level 을 붙인 전체 데이터	나중에 학습 데이터 또는 보고서 근거
scenario_front_approach.csv	정면 접근 위험 시나리오	Jetson replay 첫 번째 입력
scenario_side_blindspot.csv	측면/사각지대 접근 시나리오	V2X 사각지대 보완 설명
scenario_safe_pass.csv	비위험 통과 시나리오	오경보 방지 설명
trajectory_map.png	전체 차량 궤적과 보행자 위치	발표 자료 그림
risk_colored_map.png	위험도별 궤적 시각화	왜 위험한지 시각화
risk_timeline.png	시간에 따른 거리/TTC/risk 변화	진동이 왜 세지는지 설명
risk_label_rule.txt	위험도 계산 규칙 문서	팀원들이 같은 기준으로 설명

6.4 최민서 현장 작업 카드 - 위험 시나리오 담당 세부 절차

C-1. CSV 원본 확인 카드

- 원본 fcd_trajectory.csv 를 04_minseo_sumo_ai 폴더에 넣는다.
- Python 으로 df.columns 를 출력한다.
- time, vehicle_id, x, y, speed 가 있는지 확인한다.
- veh_id, id, car_id 처럼 이름이 다르면 vehicle_id 로 rename 한다.
- 행 수와 차량 수를 출력한다. 차량이 너무 적어도 더미 시나리오로 보완할 수 있다.
- 원본 CSV 는 절대 덮어쓰지 않고, 결과는 minseo_outputs 폴더에 저장한다.

C-2. 보행자 기준점 설정 카드

- 처음에는 실제 GPS 가 아니라 고정 보행자 좌표 PED_X, PED_Y 를 사용한다.
- 차량 궤적과 너무 멀면 risk 가 전부 0 이 되므로, 궤적 근처에 둔다.
- 차량 시작점이 너무 가까우면 risk 가 처음부터 3 이 되므로, 시작 거리도 확인한다.
- 정면 접근 시나리오는 시간이 지날수록 distance 가 줄어야 한다.
- 측면 접근 시나리오는 x 또는 y 한 축이 보행자 근처로 접근하도록 만든다.
- 비위험 통과 시나리오는 보행자와 일정 거리 이상 떨어져 지나가게 만든다.

C-3. risk 계산 검증 카드

- distance_to_ped 최소값과 최대값을 출력한다.
- ttc 가 전부 999 라면 접근 속도 계산이 잘못되었거나 차량이 접근하지 않는 것이다.
- risk_level value_counts 를 출력한다.
- risk 3 이 하나도 없으면 시연에서 위험 상황이 약하다. 보행자 좌표를 조정한다.
- risk 0 이 하나도 없으면 항상 울리는 시스템처럼 보인다. safe_pass 를 반드시 만든다.
- 최종적으로 risk 0/1/2/3 이 모두 존재하는 front 또는 side 시나리오를 확보한다.

C-4. 시각화 생성 카드

1. trajectory_map.png 에는 차량 궤적과 보행자 위치가 보여야 한다.
2. risk_colored_map.png 에는 risk_level 에 따라 위험 구간이 구분되어야 한다.
3. risk_timeline.png 에는 시간에 따라 distance 또는 TTC 가 줄고 risk 가 올라가는 흐름이 보여야 한다.
4. 그래프 제목과 축 이름을 넣는다. 발표 때 축을 설명할 수 있어야 한다.
5. PNG 파일은 최종 폴더와 발표자료 폴더에 모두 복사한다.
6. 그래프가 이상하면 x/y 단위, NaN, 정렬 순서, vehicle_id 그룹화를 확인한다.

C-5. 강현준에게 넘기는 카드

1. 가장 먼저 scenario_front_approach.csv 를 넘긴다. 이 파일이 risk replay 1 차 시연용이다.
2. 그 다음 side_blindspot, safe_pass 를 넘긴다.
3. 파일을 넘길 때 컬럼 이름이 time, risk_level, distance_to_ped, ttc 인지 말해준다.
4. 강현준이 replay 를 돌릴 때 터미널의 risk 변화와 본인의 risk_timeline 이 맞는지 같이 본다.
5. 박중선 출력부가 risk 단계별로 반응하면, 최민서는 “데이터 기반 위험도 -> 물리 피드백 연결 성공”이라고 기록한다.
6. 최종적으로 risk_label_rule.txt 에 기준을 적고 발표 멘트를 준비한다.

7. End-to-End 통합 시연 절차

7.1 Mode A - 민서 CSV replay 기반 안정 시연

이 모드는 실제 차량이 없어도 위험 상황을 안정적으로 반복 재현할 수 있다. 발표 때 가장 먼저 보여주기 좋다.

1. 최민서가 scenario_front_approach.csv 를 만든다.
2. 강현준이 Jetson 또는 노트북에서 risk_replay_from_csv.py 를 실행한다.
3. 수신기/출력부 ESP32 는 RISK 명령을 받을 수 있는 코드를 올려둔다.
4. 터미널에 RISK,0 / RISK,1 / RISK,2 / RISK,3 이 시간 순서대로 출력되는지 본다.
5. 박중선은 risk 단계가 바뀔 때 진동과 음성이 실제로 바뀌는지 확인한다.
6. 최민서는 risk_timeline.png 를 화면에 띄우고 거리/TTC 가 줄면서 risk 가 올라간다고 설명한다.
7. 촬영자는 터미널, 출력부, 그래프가 가능하면 한 화면에 들어오도록 찍는다.

7.2 Mode B - 실제 ESP32 송신기 기반 하드웨어 통신 시연

1. 수신기 ESP32 를 먼저 켜다. Receiver ready 를 확인한다.
2. Jetson listener 를 실행한다.
3. 송신기 ESP32 를 켜다.
4. 송신기 Serial Monitor 에서 [SEND] seq 가 증가하는지 본다.
5. 수신기 Serial Monitor 에서 [RX] seq 가 증가하는지 본다.
6. Jetson 터미널에서 seq 가 증가하는지 본다.
7. v2x_uart_log.csv 줄 수가 증가하는지 확인한다.
8. 송신기를 살짝 흔들어 IMU 값이 CSV 에도 변화하는지 본다.
9. 가능하면 야외에서 GPS valid=1 도 시도하되, GPS 때문에 전체 성공을 멈추지는 않는다.

7.3 최종 촬영 컷리스트

컷	내용	담당	성공 기준
1	전체 시스템 배치: 송신기, 수신기/출력부, Jetson, 노트북, 배터리	전원	구조가 한 화면에 보임
2	Jetson 배터리 연결: 보조배터리, DC 케이블, J48, 전원 LED	박중선	배터리로 켜진 근거가 보임
3	민서 위험 지도/risk timeline 화면	최민서	왜 올리는지 설명 가능
4	CSV replay 터미널: RISK 0-3 출력	강현준	risk 변화가 보임
5	진동/음성 출력부 클로즈업	박중선	단계별 반응 구분
6	실제 송신기 모드: [SEND] -> [RX] -> Jetson CSV	강현준	seq 와 CSV 증가
7	외관 정리: 고정판/박스/라벨링	박중선	프로토타입이 정리돼 보임
8	팀 회고와 다음 단계	전원	6차 작업이 명확함

8. 코드 묶음

아래 코드는 현장에서 바로 복사해 빠르게 쓸 수 있는 형태다. 기존 working 코드가 이미 있다면 기존 코드를 우선하고, 아래 코드는 final 로 합치기 전 test 파일에서 검증한다.

8.1 Jetson listener - JSON 수신 후 CSV 저장

```
# jetson_listener_20260517_final.py
# 목적: ESP32 수신기에서 넘어오는 JSON 한 줄을 읽고 CSV 로 저장한다.
# 사용: python3 jetson_listener_20260517_final.py /dev/ttyTHS1
import sys, json, time, serial
from datetime import datetime
from pathlib import Path
PORT = sys.argv[1] if len(sys.argv) > 1 else "/dev/ttyTHS1"
BAUD = 115200
LOG_PATH = Path("v2x_uart_log.csv")
HEADER = ["pc_time", "seq", "node", "risk", "gps_valid", "lat", "lng", "speed", "heading",
          "ax", "ay", "az", "gx", "gy", "gz", "tx_ms", "rx_ms"]
if not LOG_PATH.exists():
    LOG_PATH.write_text(", ".join(HEADER) + "\n", encoding="utf-8")
ser = serial.Serial(PORT, BAUD, timeout=0.05, write_timeout=0)
print(f"[Jetson] listening {PORT} @ {BAUD}")
buf = ""
last_print = time.time()
with LOG_PATH.open("a", encoding="utf-8") as f:
    while True:
        data = ser.read(4096)
        if not data:
            time.sleep(0.002)
            continue
        buf += data.decode("utf-8", errors="ignore")
        while "\n" in buf:
            line, buf = buf.split("\n", 1)
            line = line.strip()
            if not line:
                continue
            try:
                d = json.loads(line)
            except json.JSONDecodeError:
                print("[JSON FAIL]", line[:120])
                continue
            row = [datetime.now().isoformat(timespec="milliseconds")]
            for key in HEADER[1:]:
                row.append(d.get(key, ""))
            f.write(", ".join(map(str, row)) + "\n")
            f.flush()
            print(f"seq={d.get('seq')} risk={d.get('risk')} gps={d.get('gps_valid')} lat={d.get('lat')} lng={d.get('lng')}")
```

8.2 CSV replay - 민서 scenario CSV 를 risk 명령으로 송신

```
# risk_replay_from_csv.py
# 목적: scenario_*.csv 를 시간 순서대로 읽고 ESP32 출력부로 RISK 명령을 보낸다.
# 사용: python3 risk_replay_from_csv.py scenario_front_approach.csv /dev/ttyUSB0
import sys, time
import pandas as pd
```

```

import serial
csv_path = sys.argv[1] if len(sys.argv) > 1 else "scenario_front_approach.csv"
port = sys.argv[2] if len(sys.argv) > 2 else "/dev/ttyUSB0"
baud = 115200
df = pd.read_csv(csv_path).sort_values("time").reset_index(drop=True)
required = ["time", "risk_level"]
for c in required:
    if c not in df.columns:
        raise ValueError(f"missing column: {c}")
ser = serial.Serial(port, baud, timeout=0, write_timeout=0)
print(f"[REPLAY] {csv_path} -> {port}")
prev_t = float(df.loc[0, "time"])
for _, row in df.iterrows():
    t = float(row["time"])
    dt = max(0.0, min(t - prev_t, 0.5))
    time.sleep(dt)
    prev_t = t
    risk = int(row["risk_level"])
    dist = float(row.get("distance_to_ped", -1))
    ttc = float(row.get("ttc", -1))
    msg = f"RISK,{risk},{t:.2f},{dist:.2f},{ttc:.2f}\n"
    ser.write(msg.encode("utf-8"))
    print(msg.strip())
print("[REPLAY DONE]")

```

8.3 ESP32 - Jetson RISK 명령 파서

```

// 수신기 ESP32 쪽에 추가하는 테스트용 명령 파서 예시
// Jetson/노트북에서 "RISK,3,2.50,1.80,0.90" 형태로 보내면 risk=3 만 뽑아 출력부에 적용한다.
void handleJetsonCommand(String line) {
    line.trim();
    if (!line.startsWith("RISK,")) return;
    int firstComma = line.indexOf(',');
    int secondComma = line.indexOf(',', firstComma + 1);
    String riskStr;
    if (secondComma > 0) riskStr = line.substring(firstComma + 1, secondComma);
    else riskStr = line.substring(firstComma + 1);
    int risk = riskStr.toInt();
    risk = constrain(risk, 0, 3);
    applyRiskOutput(risk);
    Serial.print("[JETSON RISK CMD] risk=");
    Serial.println(risk);
}

void loop() {
    // 기존 ESP-NOW 수신 로직 유지
    // Jetson 명령 수신은 Serial2 또는 USB Serial 중 실제 연결에 맞게 선택
    while (Serial2.available()) {
        String line = Serial2.readStringUntil('\n');
        handleJetsonCommand(line);
    }
}

```

8.4 ESP32 - risk 출력 함수 빼대

// 실제 핀 번호는 박중선 핀맵과 기존 working 코드에 맞춘다.

```
const int VIB_PIN = 26;
void applyRiskOutput(int risk) {
  if (risk == 0) {
    analogWrite(VIB_PIN, 0);
    // 필요하면 DFPlayer 정지 또는 무음 유지
  } else if (risk == 1) {
    analogWrite(VIB_PIN, 80); // 약한 진동
    // dfplayer.play(1);    // "주의하세요"
  } else if (risk == 2) {
    analogWrite(VIB_PIN, 160); // 중간 진동
    // dfplayer.play(2);    // "경고"
  } else {
    analogWrite(VIB_PIN, 255); // 강한 진동
    // dfplayer.play(3);    // "위험합니다. 즉시 정지하세요"
  }
}
```

8.5 최민서 위험 시나리오 생성 코드

```
# build_risk_scenarios.py
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
SRC = Path("fcd_trajectory.csv")
OUT = Path("minseo_outputs")
OUT.mkdir(exist_ok=True)
if SRC.exists():
    df = pd.read_csv(SRC)
else:
    t = np.arange(0, 10.1, 0.1)
    df = pd.DataFrame({"time": t, "vehicle_id": "car_01", "x": 30 - 2.8*t,
                       "y": 10.0, "speed": 2.8, "angle": 180.0})
# 컬럼명 보정 예시
if "id" in df.columns and "vehicle_id" not in df.columns:
    df = df.rename(columns={"id": "vehicle_id"})
required = ["time", "vehicle_id", "x", "y", "speed"]
for c in required:
    if c not in df.columns:
        raise ValueError(f"missing column: {c}")
df = df.sort_values(["vehicle_id", "time"]).reset_index(drop=True)
PED_X, PED_Y = 5.0, 10.0
def add_risk_columns(g):
    g = g.copy().sort_values("time")
    dx = g["x"] - PED_X
    dy = g["y"] - PED_Y
    g["ped_x"] = PED_X
    g["ped_y"] = PED_Y
    g["distance_to_ped"] = np.sqrt(dx*dx + dy*dy)
    dt = g["time"].diff().fillna(0.1).replace(0, 0.1)
    dd = -g["distance_to_ped"].diff().fillna(0)
```



```

g["relative_speed"] = (dd / dt).clip(lower=0)
g["ttc"] = np.where(g["relative_speed"] > 0.1, g["distance_to_ped"] / g["relative_speed"], 999.0)
risk = np.zeros(len(g), dtype=int)
risk[(g["distance_to_ped"] < 10) | (g["ttc"] < 5)] = 1
risk[(g["distance_to_ped"] < 6) | (g["ttc"] < 3)] = 2
risk[(g["distance_to_ped"] < 3) | (g["ttc"] < 1.5)] = 3
g["risk_level"] = risk
return g

risk_df = pd.concat([add_risk_columns(g) for _, g in df.groupby("vehicle_id")], ignore_index=True)
risk_df.to_csv(OUT / "risk_labeled_fcd.csv", index=False)
print(risk_df["risk_level"].value_counts().sort_index())
best_vid = risk_df.groupby("vehicle_id")["risk_level"].max().sort_values(ascending=False).index[0]
front = risk_df[risk_df["vehicle_id"] == best_vid].copy()
front["scenario"] = "front_approach"
front.to_csv(OUT / "scenario_front_approach.csv", index=False)
side_t = np.arange(0, 10.1, 0.1)
side = pd.DataFrame({"time": side_t, "vehicle_id": "side_car_01", "x": PED_X + 1.0,
                    "y": PED_Y + 22 - 2.2*side_t, "speed": 2.2, "angle": 270.0})
side = add_risk_columns(side); side["scenario"] = "side_blindspot"
side.to_csv(OUT / "scenario_side_blindspot.csv", index=False)
safe_t = np.arange(0, 10.1, 0.1)
safe = pd.DataFrame({"time": safe_t, "vehicle_id": "safe_car_01", "x": PED_X + 20 + 2.0*safe_t,
                    "y": PED_Y + 15, "speed": 2.0, "angle": 0.0})
safe = add_risk_columns(safe); safe["scenario"] = "safe_pass"
safe.to_csv(OUT / "scenario_safe_pass.csv", index=False)
plt.figure(figsize=(7,5))
for vid, g in risk_df.groupby("vehicle_id"):
    plt.plot(g["x"], g["y"], alpha=0.5)
plt.scatter([PED_X], [PED_Y], s=120, marker="*", label="pedestrian")
plt.title("Vehicle trajectories and pedestrian point")
plt.xlabel("x"); plt.ylabel("y"); plt.axis("equal"); plt.legend(); plt.tight_layout()
plt.savefig(OUT / "trajectory_map.png", dpi=140)
plt.figure(figsize=(7,5))
sc = plt.scatter(risk_df["x"], risk_df["y"], c=risk_df["risk_level"], s=8)
plt.scatter([PED_X], [PED_Y], s=120, marker="*", label="pedestrian")
plt.title("Risk colored trajectory map")
plt.xlabel("x"); plt.ylabel("y"); plt.axis("equal"); plt.colorbar(sc, label="risk_level")
plt.tight_layout(); plt.savefig(OUT / "risk_colored_map.png", dpi=140)
plt.figure(figsize=(8,4))
plot_df = front.head(150)
plt.plot(plot_df["time"], plot_df["distance_to_ped"], label="distance")
plt.plot(plot_df["time"], plot_df["ttc"].clip(upper=10), label="ttc clipped")
plt.step(plot_df["time"], plot_df["risk_level"], label="risk", where="post")
plt.title("Risk timeline"); plt.xlabel("time (s)"); plt.legend(); plt.tight_layout()
plt.savefig(OUT / "risk_timeline.png", dpi=140)
rule = ""
risk 0: 차량이 멀거나 보행자 방향으로 접근하지 않음
risk 1: distance < 10m 또는 TTC < 5 초
risk 2: distance < 6m 또는 TTC < 3 초
risk 3: distance < 3m 또는 TTC < 1.5 초
""

(OUT / "risk_label_rule.txt").write_text(rule, encoding="utf-8")

```

print("done", OUT)

9. 문제 해결표

문제	가장 흔한 원인	누가 처리	해결 순서
송신기 [SEND] 안 뜸	포트/보드 선택, USB 케이블, 업로드 실패	박중선	COM 포트 확인 -> 케이블 교체 -> working 코드 재업로드 -> 보드 리셋
수신기 [RX] 안 뜸	MAC 주소 불일치, 구조체 불일치, 송수신 거리/전원 문제	박중선/강현준	수신기 MAC 출력 확인 -> sender MAC 수정 -> typedef 동일 복사 -> 가까이 두고 테스트
Jetson 에 글자가 안 들어옴	TX/RX 반대, GND 없음, 포트명 오류, nvgetty	강현준	GND 공유 -> TX/RX 교차 -> /dev 포트 변경 -> nvgetty 중지 -> USB Serial 대체
JSON FAIL 반복	ESP32 디버그 문구가 JSON 포트에 섞임	강현준	Jetson 으로 보내는 포트에는 JSON 만 출력 -> 디버그는 USB Serial 로 분리
CSV 줄 수가 안 늘어남	경로/권한/flush 문제	강현준	현재 폴더 확인 -> 파일 권한 확인 -> f.flush() 확인 -> tail -f 실행
모터 작동 시 ESP32 리셋	브라운아웃, 전원 부족	박중선	모터 전원 분리 -> GND 공통 -> 캐패시터 추가 -> PWM 낮춤
DFPlayer 무음	SD 카드 파일명, 전원, TX/RX, 볼륨	박중선	/mp3/0001.mp3 확인 -> 5V 전원 -> 스피커 -> TX/RX -> 볼륨 코드
Jetson 배터리로 안 켜짐	J48 점퍼 없음, 케이블 전압/극성 오류, 전류 부족	박중선	J48 확인 -> 멀티미터 5V 확인 -> DC Jack 규격 확인 -> 어댑터로 비교
GPS valid=0	실내, cold start, 안테나 가림	박중선	실패로 보지 말고 야외 3-10 분 대기 -> 안테나 위쪽 -> 로그만 확보
민서 risk 가 전부 0	보행자 위치가 너무 멀거나 접근 속도가 0	최민서	PED_X/Y 를 궤적 근처로 이동 -> side/front 더미 시나리오 사용
민서 risk 가 전부 3	시작부터 너무 가까움, 임계값 과민	최민서	safe_pass 확인 -> 시작 거리 늘림 -> 임계값 조정
GitHub 꼬임	동일 파일 동시 수정, OneDrive 경로 문제	강현준	working/final 파일명 분리 -> pull 먼저 -> 충돌 파일 공유

10. 최종 산출물 및 다음 회차 연결

10.1 최종 백업 폴더 구조

2026_05_17_5th_meeting_final/

- ├── 01_sender_esp32/
 - | ├── v2x_sender_20260517_working.ino
 - | ├── v2x_sender_20260517_test_risk_force.ino
 - | └── v2x_sender_20260517_final.ino
- ├── 02_receiver_esp32/
 - | ├── v2x_receiver_20260517_working.ino
 - | ├── v2x_receiver_20260517_uart_risk_cmd.ino
 - | └── v2x_receiver_20260517_final.ino
- ├── 03_jetson/
 - | ├── jetson_listener_20260517_final.py
 - | ├── risk_replay_from_csv.py
 - | └── v2x_uart_log_20260517.csv
- ├── 04_minseo_sumo_ai/
 - | ├── scenario_front_approach.csv
 - | ├── scenario_side_blindspot.csv
 - | └── scenario_safe_pass.csv

- | |—— risk_labeled_fcd.csv
- | |—— trajectory_map.png
- | |—— risk_colored_map.png
- | |—— risk_timeline.png
- | |—— risk_label_rule.txt
- |—— 05_photos/
- | |—— jetson_battery_j48_dcjack.jpg
- | |—— sender_wiring.jpg
- | |—— receiver_actuator_wiring.jpg
- | |—— prototype_layout.jpg
- |—— 06_videos/
- | |—— e2e_demo_20260517_1min.mp4
- | |—— risk_output_0_to_3.mp4
- | |—— jetson_csv_logging.mp4
- |—— README_result.txt

10.2 README_result.txt 양식

[5.17 5 차 통합 미팅 결과]

날짜: 2026.05.17

참여자: 강현준, 박중선, 최민서

작업 시간:

1. 최종 성공 항목

- 송신기 [SEND]: 성공 / 실패 / 부분 성공
- 수신기 [RX]: 성공 / 실패 / 부분 성공
- Jetson 전원: 어댑터 성공 / 배터리 성공 / 배터리 미완
- Jetson UART/USB Serial: 성공 포트 =
- CSV 저장: v2x_uart_log.csv 줄 수 =
- 민서 CSV replay: 성공한 scenario =
- risk 진동/음성 출력: risk 0/1/2/3 확인 여부 =
- GPS Fix: 실내 valid=0 / 야외 valid=1 / 미확인
- 외관 정리: 고정판 / 박스 / 라벨링 / 미완
- End-to-End 영상: 파일명 =

2. 미완성 항목

-

3. 발생한 문제와 해결

- 문제:
- 원인:
- 해결:

4. 최종 사용 파일

- Sender final:
- Receiver final:
- Jetson listener:
- risk replay:
- Minseo scenario CSV:
- CSV log:
- 영상 파일:

5. 다음 회차 첫 작업

-

10.3 다음 회차로 넘길 것

다음 회차 작업	5 차에서 준비해야 하는 것	설명
지팡이형 v1 외형	핀맵, 전원 구조, 배선 사진, 박스 정리안	5 차 연결이 끝나야 빠르게 패키징 가능

Jetson ONNX/실시간 추론	민서 risk_labeled 데이터, 모델 인터페이스	rule 기반 risk 를 모델 출력으로 확장
RC Car 또는 차량 송신기	v2x_message_t 공통 구조체와 송신 코드	차량 노드 역할 구현
발표자료	위험 지도 이미지, 통합 영상, Jetson 배터리 사진	공모전 설득력 확보
보고서	README_result, 실패/해결 기록	기술적 신뢰도 확보

10.4 마지막 최종 체크리스트

담당	체크	완료 조건
박중선	☐ 송신기 [SEND] seq 증가	1 분 이상 영상/캡처
박중선	☐ 수신기 [RX] seq 증가	1 분 이상 영상/캡처
박중선	☐ risk 0-3 진동/음성 출력	단계별 영상
박중선	☐ Jetson 배터리 연결 확인	J48, DC Jack, 보조배터리 사진
박중선	☐ 고정판/박스/라벨링 적용	외관 사진
강현준	☐ Jetson seq 출력	터미널 캡처
강현준	☐ v2x_uart_log.csv 증가	wc -l 캡처
강현준	☐ 민서 CSV replay 로 risk 출력 연결	터미널+출력부 영상
강현준	☐ GitHub final 업로드	GitHub Desktop 캡처
최민서	☐ scenario CSV 3 개 생성	CSV 파일
최민서	☐ distance/TTC/risk 계산	risk_labeled_fcd.csv
최민서	☐ PNG 3 개 생성	trajectory/risk/timeline 이미지
전원	☐ 1 분 이상 통합 영상	e2e_demo mp4
전원	☐ README_result 작성	최종 폴더에 저장

이번 5 차의 목표는 “완제품”이 아니라 “완제품으로 묶기 전 마지막 연결 검증”이다.
 박중선은 하드웨어와 전원 안정성, 강현준은 전체 연결성, 최민서는 위험도의 데이터 근거를 담당한다.
 이 세 가지가 끝나면 하드웨어 v0 는 공모전 시연 가능한 1 차 완성으로 볼 수 있다.

7.4 현장 중단/전환 의사결정표

시간 또는 상황	그대로 계속하면 생기는 문제	전환 행동	성공으로 인정할 증거
시작 후 1 시간인데 ESP-NOW 가 안 됨	전체 연결 시간이 사라짐	새 기능 금지. MAC/구조체/working 코드 복구만 한다.	송신기 [SEND] 또는 수신기 [RX] 중 하나라도 영상 확보
시작 후 2 시간인데 Jetson 헤더 UART 가 안 됨	UART 만 붙잡다가 CSV/replay 를 못 함	USB Serial 로 전환하고 헤더 UART 는 6 차 과제로 기록	/dev/ttyUSB0 로 CSV 저장 성공
시작 후 3 시간인데 민서 risk CSV 가 안 됨	출력부 시연 입력이 없어짐	더미 front/side/safe 시나리오 생성 사용	scenario_front_approach.csv 생성
시작 후 4 시간인데 DFPlayer 가 무음	음성 때문에 전체 시연이 막힘	진동 출력만 확실히 하고 음성은 실패 원인 기록	risk 0-3 진동 영상
시작 후 5 시간인데 외관이 미완	촬영 증거가 부족해짐	외관보다 영상/CSV/코드 백업 우선	1 분 이상 E2E 영상
종료 40 분 전	새 수정은 실패 가능성이 큼	코드 변경 중단, 촬영/백업/README 작성	최종 폴더 완성

7.5 발표/보고서에 바로 쓸 수 있는 설명 문장

상황	말할 문장
전체 시스템 설명	저희 시스템은 ESP32 기반 송신 노드가 보행자 또는 차량 관련 데이터를 수집 하고, 수신 노드가 이를 Jetson 엣지 노드로 전달한 뒤, 위험도 계산 결과를 진동과 음성으로 사용자에게 제공하는 구조입니다.
Jetson 을 지팡이에 붙이지 않는 이유	Jetson Nano 는 지팡이 본체에 반드시 부착되는 장치라기보다, 스마트시티 환

	경의 노변 엣지 중계기 또는 근거리 RSU 역할로 돌 수 있습니다. 지팡이는 사용자 단말이고, Jetson 은 주변 인프라에서 계산을 담당할 수 있습니다.
CSV replay 를 쓰는 이유	실제 차량을 반복적으로 이용한 위험 실험은 위험하고 재현성이 낮기 때문에, 먼저 SUMO/CSV 기반 시나리오를 사용하여 같은 위험 상황을 반복 재생하고 출력부 반응을 검증했습니다.
외관이 완제품이 아닌 이유	현재는 기능 검증을 우선한 프로토타입 단계입니다. 브레드보드 회로를 고정판과 케이스로 정리하여 시연 안정성을 확보했고, 이후 PCB 또는 소형 케이스로 통합할 계획입니다.
박중선 한 문장	저는 송신기/수신기 하드웨어와 위험도별 진동·음성 출력부를 안정화하고, Jetson 배터리 연결과 전원 안정성까지 확인했습니다.
강현준 한 문장	저는 송신기, 수신기, Jetson, CSV 저장, risk replay, GitHub 정리까지 전체 연결 흐름을 담당했습니다.
최민서 한 문장	저는 차량 궤적 CSV를 보행자 기준 거리와 TTC 로 분석하여 risk_level 0-3 을 계산하고, Jetson 에서 재생 가능한 위험 시나리오와 시각화 자료를 만들었습니다.

7.6 6 시간 현장 기록지

00:00-01:00 준비/4 차 재현

항목	기록
성공한 것	
실패한 것	
바꾼 코드 파일명	
촬영한 영상/사진	
다음 시간으로 넘길 문제	

01:00-02:00 Jetson 전원/통신

항목	기록
성공한 것	
실패한 것	
바꾼 코드 파일명	
촬영한 영상/사진	
다음 시간으로 넘길 문제	

02:00-03:00 CSV replay/risk 출력

항목	기록
성공한 것	
실패한 것	
바꾼 코드 파일명	
촬영한 영상/사진	
다음 시간으로 넘길 문제	

03:00-04:00 실제 송신기 모드

항목	기록
성공한 것	
실패한 것	

바꾼 코드 파일명	
촬영한 영상/사진	
다음 시간으로 넘길 문제	

04:00-05:00 외관/전원 안정화

항목	기록
성공한 것	
실패한 것	
바꾼 코드 파일명	
촬영한 영상/사진	
다음 시간으로 넘길 문제	

05:00-06:00 촬영/백업

항목	기록
성공한 것	
실패한 것	
바꾼 코드 파일명	
촬영한 영상/사진	
다음 시간으로 넘길 문제	

10.5 최종 제출 전 품질 점검

점검 항목	확인 질문	통과 기준
문서/코드	final 코드와 test 코드가 분리되어 있는가?	final 에는 강제 테스트가 섞이지 않음
전원	Jetson 배터리 연결 사진이 있는가?	J48/DC/보조배터리가 보이는 사진
하드웨어	선이 빠져도 복구할 핀맵 사진이 있는가?	송신기/수신기/출력부 배선 사진
데이터	CSV 로그가 실제로 증가했는가?	wc -l 캡처
위험도	risk 가 0-3 으로 바뀐 근거가 있는가?	risk_timeline.png 와 터미널 replay
피드백	진동/음성이 risk 와 연결되었는가?	risk 단계별 영상
발표성	프로토타입이 너무 지저분해 보이지 않는가?	고정판/라벨링 사진
재현성	다음 회차에 10 분 안에 다시 켤 수 있는가?	README_result 와 파일명 정리